Reproducible Data Analysis

STAT 184 Spring 2026

Naman Joshi

2026-04-15

1 Busiest Airport Analysis

The first section focuses on passenger traffic data for six major airports: Atlanta (ATL), Frankfurt (FRA), Los Angeles (LAX), Dubai (DXB), Dallas (DFW), and Tokyo Haneda (HND), covering the years 2020 to 2025. The data were used to create both a table and a line graph to compare passenger counts across these airports over time.

The table and the line graph together provide a clear comparison of passenger traffic trends. The table presents the exact passenger counts for each airport and year, while the graph highlights overall patterns and makes it easier to visually identify trends. As shown in Table 1 and Figure 1, all airports experienced a noticeable decline in passenger traffic around 2020, which aligns with the global impact of COVID-19 and reduced travel during that period. This decline is followed by steady recovery and growth in the following years.

Hartsfield–Jackson Atlanta International Airport consistently remains the busiest airport across all years, with passenger traffic exceeding 100 million by 2023 and continuing to remain high afterward. Dubai International Airport and Tokyo Haneda Airport show strong growth trends, especially after 2021, indicating increasing travel demand in those regions. On the other hand, Frankfurt Airport has the lowest passenger traffic among the six and shows relatively slower growth over time.

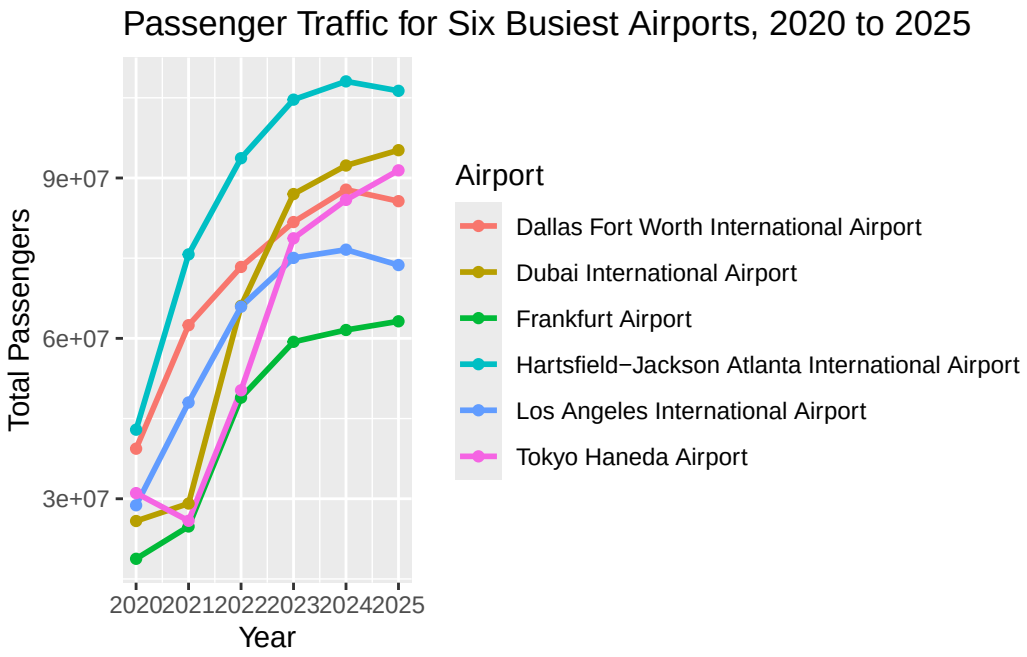
The line graph makes it easier to observe these changes, such as the sharp increases between 2021 and 2023 for most airports, while the table helps confirm the exact values and differences between airports and years. Together, these visualizations demonstrate how a global event like a pandemic can significantly impact travel patterns and how the industry can recover over time, in some cases reaching or even exceeding pre-pandemic levels.

Overall, the data suggest that global air travel has increased significantly since 2020, with some airports, particularly Atlanta and Dubai, leading in total passenger traffic, while others are growing at a more moderate pace.

Table 1: Passenger traffic for six major airports, 2020 to 2025

airport	2020	2021	2022	2023	2024	2025
Dallas Fort Worth (DFW)	39,364,990	62,465,756	73,362,946	81,755,538	87,817,864	85,660,127
Dubai (DXB)	25,836,771	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
Frankfurt (FRA)	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
Atlanta (ATL)	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
Los Angeles (LAX)	28,779,527	48,007,284	65,924,298	75,050,875	76,588,028	73,709,594
Tokyo Haneda (HND)	31,055,210	25,876,429	50,290,705	78,719,302	85,900,167	91,430,000

Figure 1: Passenger traffic trends for six major airports, 2020 to 2025



2 Monte Carlo Numerical Integration

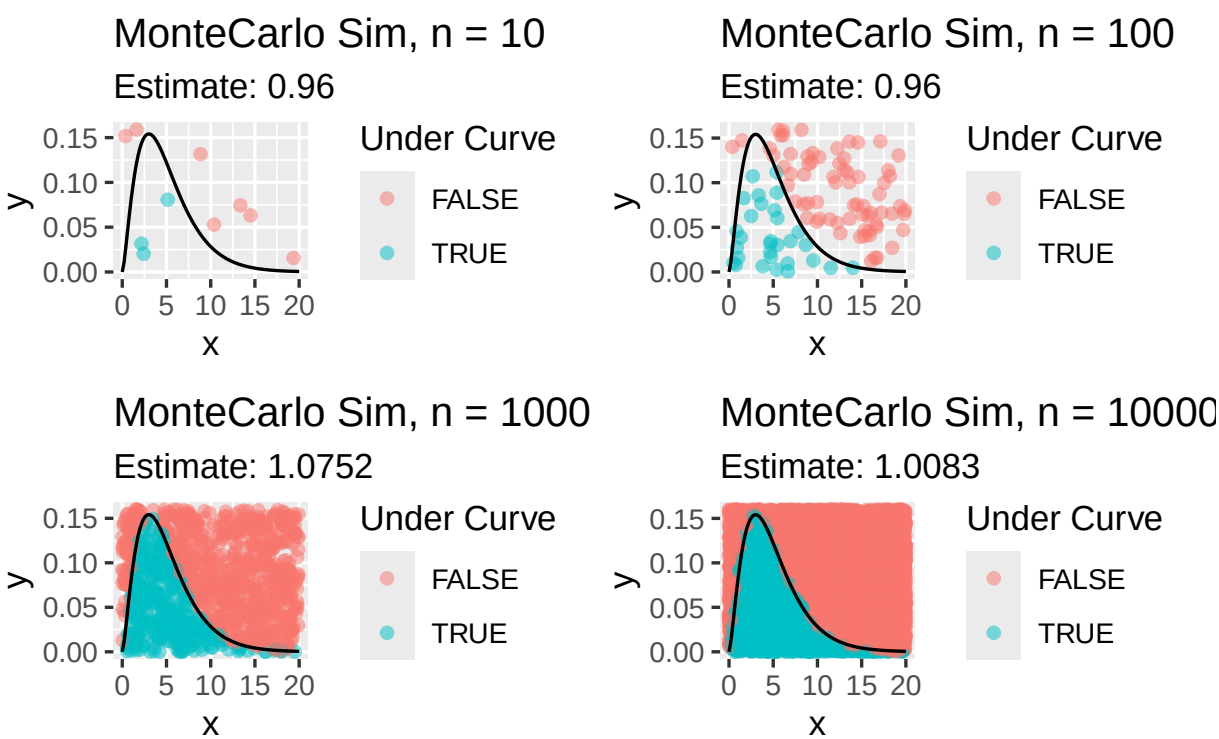
This section revisits the Monte Carlo numerical integration activity from HW #3.4. Monte Carlo integration estimates the area under a curve by using randomly generated points, and the accuracy of the estimate generally improves as the sample size increases.

The small-multiple visualization in Figure 2 shows how the estimate changes as the number of simulated points increases. With smaller sample sizes, the random points are more unevenly distributed, which leads to greater variability in the estimate. As the sample size increases, the points become more evenly spread across the region, and the estimate stabilizes.

For the function used in this assignment, the Monte Carlo estimate approaches a value close to 1 as the number of points increases. This makes sense because we are effectively integrating a probability density function, and the total area under a PDF is equal to 1. The visualization clearly demonstrates how increasing the number of simulated points improves the accuracy and reliability of the estimate.

Overall, the small multiple plots effectively show how simulation-based methods behave. With fewer points, the estimate is less reliable due to randomness, while larger sample sizes reduce variation and produce results that are closer to the true value. This reinforces the idea that increasing sample size leads to better approximations in Monte Carlo methods.

Figure 2: Monte Carlo integration results across multiple sample sizes



3 Planning and Prompting GenAI Tools

3.1 My Plan

Nouns: dataset structure, time, null group, treatment group, group identifiers, numeric, observations, dataset, variables, calcium measurements, tidy data structure, visualization

Verbs: understand, identify,recognize, separate, reshape, create, add, clean, ensure, prepare, convert, visualize, compare, interpret

Plan (Step-by-Step Approach):

1. Understand the dataset structure

- Recognize that the dataset is in wide format
- Identify that the first four columns correspond to the null group and the next four correspond to the treatment group

2. Reshape the data

- Use pivot_longer() to convert the dataset into long format
- Create a single column for time (Year 0, Year 1, Year 2, Year 3)

3. Create group identifiers

- Add a new variable indicating whether each observation belongs to the null group or treatment group

4. Clean the data

- Ensure all calcium measurements are numeric
- Organize the dataset so each row represents one observation at one time point

5. Prepare variables for plotting

- Convert time into an ordered factor
- Ensure group labels are clear and readable

6. Create the visualization

- Use ggplot2
- Plot time on the x-axis and calcium measurements on the y-axis
- Use color to distinguish between the two groups
- Add summary lines (mean trends) for each group

7. Interpret the results

- Compare how calcium levels change over time between groups
- Identify any differences or trends between the null and treatment groups

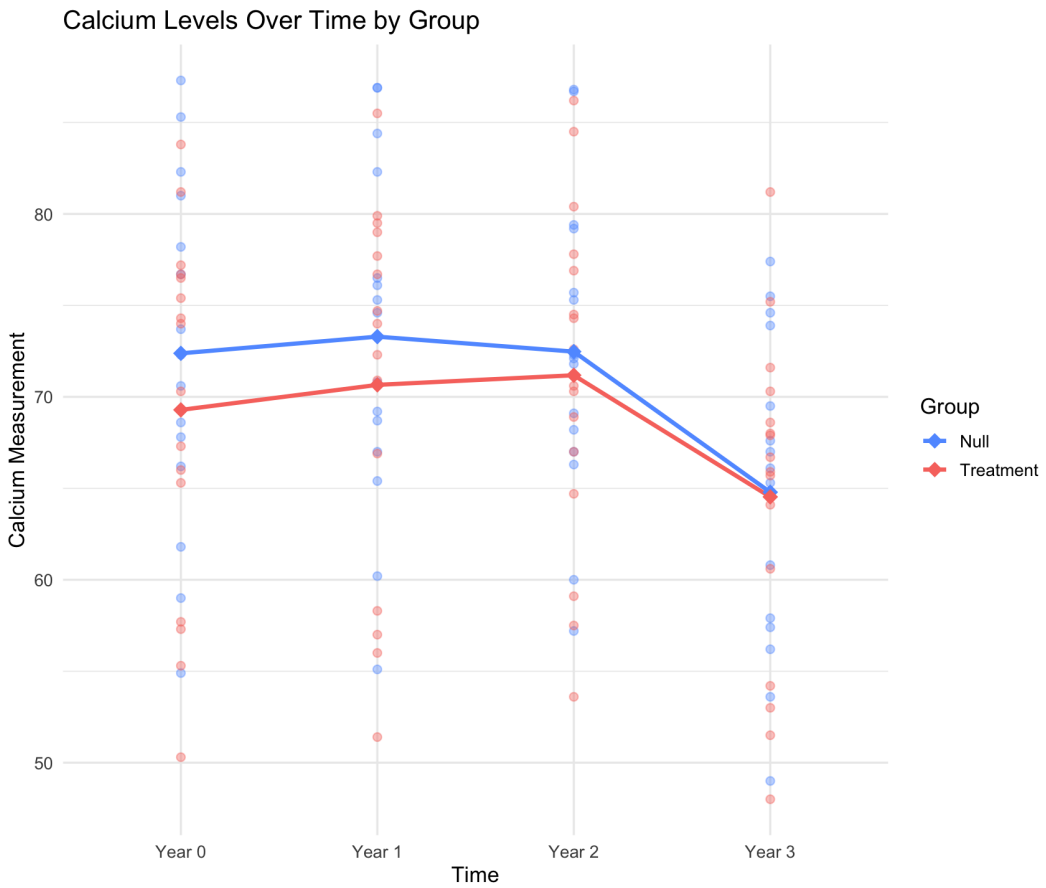
3.2 Plan-Informed Response

I used Anthropic Claude and provided my plan together with the original calcium.csv file. Claude returned R code that reads the data, adds an ID variable, reshapes the null and treatment groups separately using `pivot_longer()`, combines the results into one tidy dataset, recodes the time values into readable labels, and creates a line plot with points and group mean trends.

Claude's response is focused on keeping the code simple and easy for an introductory R course. The response used `tidyverse`, created a long-format dataset called `calcium_long`, and used `ggplot2` to compare calcium measurements across time for the two treatment groups.

I provided my structured plan along with the dataset to Anthropic Claude. The response generated included R code to reshape the dataset, create a tidy format, and produce a visualization comparing calcium levels across time for both groups.

Figure 3: Claude plan-informed response showing R code and explanation



3.3 Generic Response

The generic response from the selected genAI tool is shown in Figure 4 and Figure 5. The first figure presents the average calcium measurements for both groups across the four time points, while the second figure shows the individual trends for each participant. From Figure 4, it is clear that both groups follow a very similar pattern over time, with a slight increase from Year 0 to Year 2 and then a noticeable decrease by Year 3. The treatment group appears to have slightly lower calcium levels overall, but the difference between the two groups is not very large. In Figure 5, we can see the variability across individuals, which helps confirm that there is a wide spread of calcium values within each group. However, the overall trend still remains consistent with the mean plot. I think the generic response does a good job of presenting the results in a clean and easy-to-understand way, especially for quickly comparing the two groups over time.

Figure 4: Generic response showing group mean trends over time

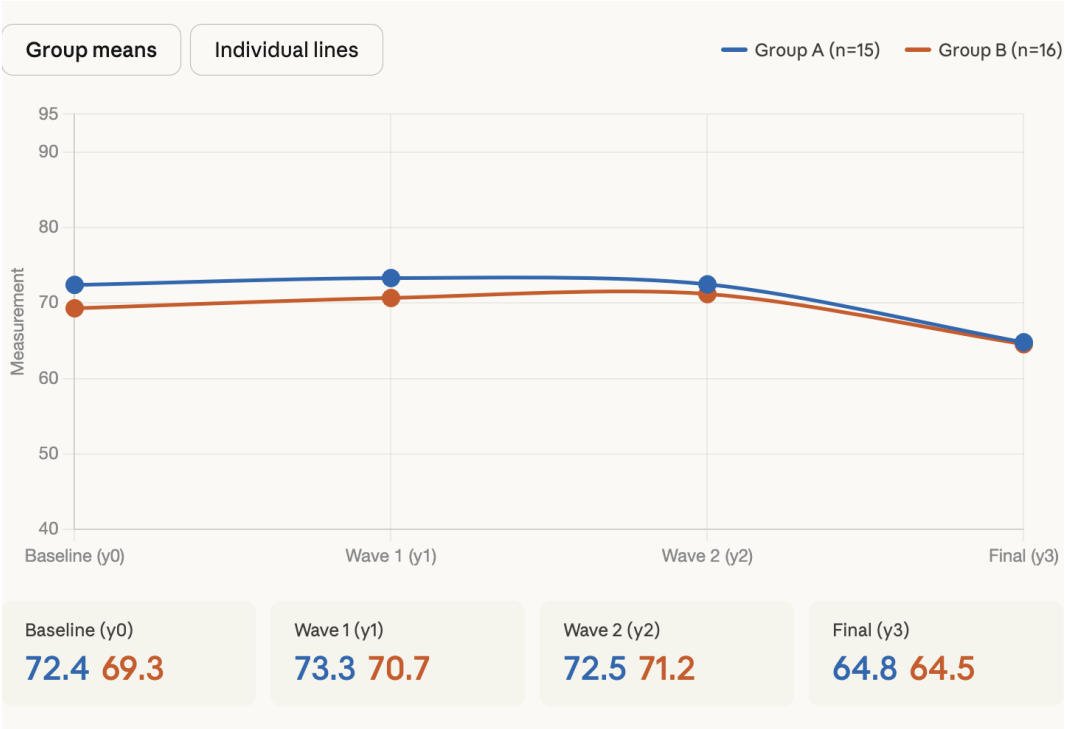
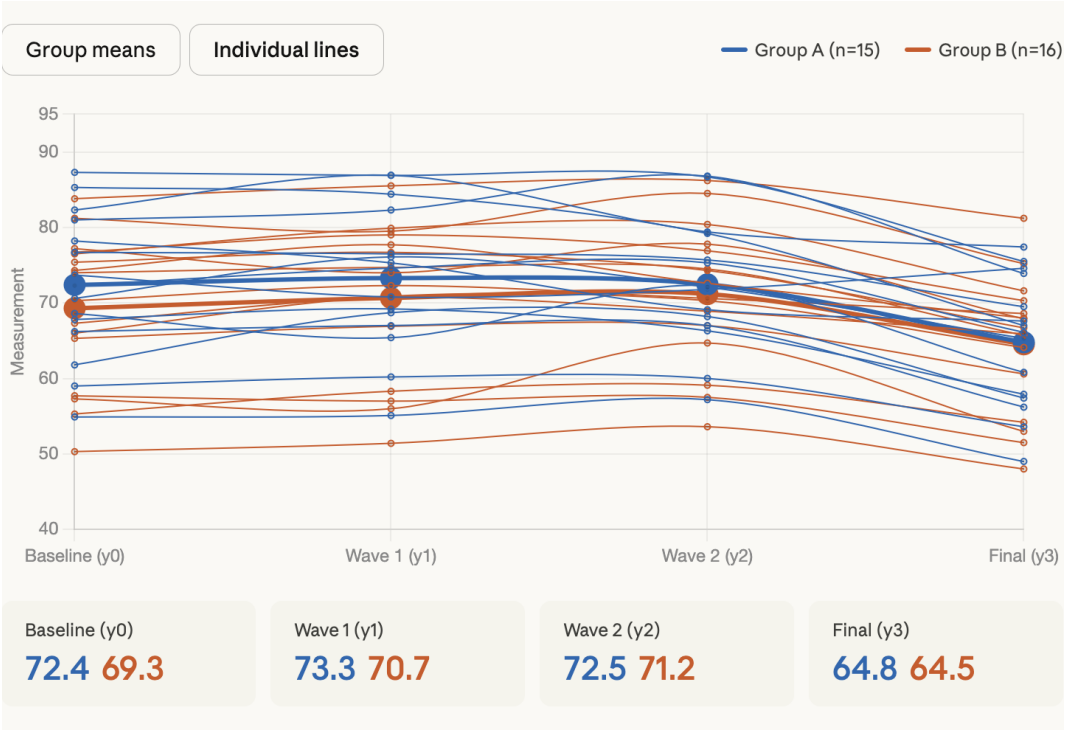


Figure 5: Generic response showing individual trajectories over time



3.4 Comparison of Plan-Informed and Generic Responses

The plan-informed response and the generic response were both useful, but they were useful in different ways. The plan-informed response was more detailed and more closely connected to the actual steps needed to tidy and organize the calcium dataset. Because I gave Claude a structured plan first, the response was more specific about reshaping the data, creating group identifiers, and preparing the variables for plotting. In that sense, the plan-informed response matched the data wrangling part of the task better and showed a clearer connection between the structure of the dataset and the code that was produced.

At the same time, the generic response produced a cleaner and easier-to-read visualization. As shown in Figure 4 and Figure 5, the generic response focused more directly on communication and made the overall trends between the two groups easier to compare visually. In contrast, the plan-informed response shown in Figure 3 included more detail, especially by showing all of the individual points along with the mean trends. That makes the output more informative, but it also makes the plot look more crowded and a little harder to read at first.

This comparison helped me understand that a more detailed prompt does not automatically lead to a better final visualization. The plan-informed response was stronger in terms of data wrangling and structure, while the generic response was stronger in terms of visual clarity. I think the main lesson from this activity is that genAI output still needs to be evaluated carefully. Even when the code is more detailed, I still have to decide whether the final graph is actually the best one for explaining the data clearly.

4 Reflection

So far in this course, I have learned that data analysis is not just about getting code to run. A big part of the work is understanding the structure of the data, deciding how to organize it, and then choosing a visualization that actually helps the reader see and understand the main point. Earlier in the semester, I was more focused on just finishing the task, but now I pay much more attention to whether the data is tidy, whether the variables are clearly defined, and whether the graph really matches the question being asked.

One thing I learned from the busiest airports assignment is how important data wrangling is before making a visualization. The original data was not immediately ready for plotting, so I had to clean the tables, keep only the airports needed for the assignment, and organize the data in a consistent way. That helped me see that even a simple line graph depends on careful preparation. I also learned that tables and graphs do different jobs. The table is useful for exact values, while the graph is better for showing patterns and trends over time.

The Monte Carlo assignment helped me understand that visualizations can also be used to explain a process, not just show final results. In that assignment, the small multiple made it much easier to see how the estimate changed as the number of points increased. That gave me a clearer understanding of why larger sample sizes usually lead to more stable estimates. It also reminded me that a graph can be effective when it directly supports the main idea instead of trying to show too much at once.

This Quarto assignment helped me understand reproducibility more clearly. Before this, I mostly thought of assignments as separate pieces like code, graphs, and written answers. In this document,

I had to combine narrative text, figures, tables, and code into one organized report. That made me think more carefully about structure, section headings, captions, and references to tables and figures. I also saw how important it is for the document to be readable for someone else, not just runnable on my own computer. I think Quarto is an awesome tool that can be used to showcase your idea in the best way possible including codes, figures and more.

The GenAI part of the assignment also taught me something useful. I noticed that giving a tool a more detailed plan can lead to a more detailed response, but that does not automatically mean the final visualization is better for communication. In this case, the plan-informed response included more information, while the generic response was actually cleaner and easier to read. That showed me that I still have to evaluate the output instead of assuming the generated result is automatically the best one. Overall, I think this course has helped me improve not only my R skills, but also the way I think about data, communication, and the purpose of a visualization.

5 Appendix A: GenAI Usage

Tool: Anthropic Claude Version: Sonnet 4.6 Date: April 14th, 2026

5.1 My Prompt:

I have a dataset called calcium.csv that contains measurements from a longitudinal study of 31 women over four years.

My plan: Plan (Step-by-Step): Understand the dataset structure

Recognize that the dataset is in wide format Identify that the first four columns correspond to the null group and the next four correspond to the treatment group Reshape the data

Use pivot_longer() to convert the dataset into long format Create a single column for time (Initial, Year 1, Year 2, Year 3) Create group identifiers

Add a new variable indicating whether each observation belongs to the null group or treatment group Clean the data

Ensure all calcium measurements are numeric Organize the dataset so each row represents one observation at one time point Prepare variables for plotting

Convert time into an ordered factor Ensure group labels are clear and readable Create the visualization

Use ggplot2 Plot time on the x-axis and calcium measurements on the y-axis Use color to distinguish between the two groups Add summary lines (mean trends) for each group Interpret the results

Compare how calcium levels change over time between groups Identify any differences or trends between the null and treatment groups The dataset is not tidy. It contains:

Two groups: a null treatment group and a diet/exercise treatment group Each group has four columns: Year 0, Year 1, Year 2, and Year 3 The first four columns correspond to the null group and the next four correspond to the treatment group I need help writing R code (using tidyverse and ggplot2) to: 1. Tidy the dataset:

Convert it from wide format to long format Create a column for time (Year 0, Year 1, Year 2, Year 3) Create a column for group (Null vs Treatment) Ensure each row represents one observation Clean and organize the data:

Make sure values are numeric Ensure time is treated as an ordered variable Create a visualization:

Use ggplot2 Plot time on the x-axis and calcium measurement on the y-axis Use color to distinguish between groups Include a line showing the mean trend for each group. Please keep the code simple and similar to what would be taught in an introductory R course.

5.2 Claude Response:

Now I have a clear picture of the data. Here's clean, intro-level R code to tidy and visualize it:

```
# Load required packages
library(tidyverse)

#   1. Read the data
calcium <- read_csv("calcium.csv")

# The first four columns are the Null group (y0-y3)
# The next four columns are the Treatment group (y0.1-y3.1)
# Each row is one woman; we add an ID column to track individuals
calcium <- calcium |>
  mutate(id = row_number())

#   2. Reshape: wide → long

# Pivot the Null group columns
null_long <- calcium |>
  select(id, y0, y1, y2, y3) |>
  pivot_longer(
    cols      = c(y0, y1, y2, y3),
    names_to  = "time",
    values_to = "calcium"
  ) |>
  mutate(group = "Null")

# Pivot the Treatment group columns
treat_long <- calcium |>
  select(id, y0.1, y1.1, y2.1, y3.1) |>
  pivot_longer(
    cols      = c(y0.1, y1.1, y2.1, y3.1),
    names_to  = "time",
    values_to = "calcium"
  ) |>
  mutate(group = "Treatment")

#   3. Combine and clean
calcium_long <- bind_rows(null_long, treat_long) |>
  # Give the time variable readable labels
  mutate(
    time = recode(time,
      "y0"   = "Year 0", "y1"   = "Year 1",
      "y2"   = "Year 2", "y3"   = "Year 3",
      "y0.1" = "Year 0", "y1.1" = "Year 1",
      "y2.1" = "Year 2", "y3.1" = "Year 3"
    ),
```

```r
    # Make time an ordered factor so it plots left → right correctly
    time  = factor(time, levels = c("Year 0", "Year 1", "Year 2", "Year 3"),
                   ordered = TRUE),
    # Make sure calcium is numeric
    calcium = as.numeric(calcium)
  )

# Quick check
glimpse(calcium_long)

#   4. Visualize
ggplot(calcium_long, aes(x = time, y = calcium, color = group)) +

  # Individual data points (slightly transparent so overlaps are visible)
  geom_point(alpha = 0.4, size = 2) +

  # Mean trend line for each group
  stat_summary(
    fun     = mean,
    geom    = "line",
    aes(group = group),
    linewidth = 1.2
  ) +

  # Mean point on top of the line
  stat_summary(
    fun   = mean,
    geom  = "point",
    size  = 4,
    shape = 18           # diamond shape for the mean
  ) +

  # Friendly colors
  scale_color_manual(
    values = c("Null" = "#619CFF", "Treatment" = "#F8766D")
  ) +

  # Labels
  labs(
    title   = "Calcium Levels Over Time by Group",
    x       = "Time",
    y       = "Calcium Measurement",
    color   = "Group"
  ) +

  theme_minimal(base_size = 13)
```

What each section does: SectionPurposemutate(id = row_number())Assigns a unique ID to each woman before reshaping so we can track individualsTwo pivot_longer() callsConverts each group separately from wide to long, then bind_rows() stacks themrecode()Translates the raw column names (y0, y1.1, …) into readable time labelsfactor(…, ordered = TRUE)Tells R that Year 0 < Year 1 < Year 2 < Year 3 so the x-axis is ordered correctlygeom_point(alpha = 0.4)Shows all 31 individual observations per group per time pointstat_summary(fun = mean, geom = "line")Draws the mean trend line — the key comparison between groups Why two separate pivot_longer() calls instead of one? Because the column names don't follow a perfectly regular pattern (y0 vs y0.1), splitting by group first and then binding is cleaner and easier to read for an intro course than a single complex pivot with names_pattern.

6 Appendix B: Code

6.1 Busiest Airports Final Code

```
# Wrangle the busiest airports data ----
## Our goals are:
## - Get the passenger traffic data from Wikipedia for 2020-2025
## - Keep only the six airports we want to compare
## - Create a tidy data set with one airport-year combination as a case
## - Make the data readable and tidy for a table and a plot


### Wrangling the data

# Step 1: Load packages ----
## Needed packages: {tidyverse}, {rvest}, {janitor}

library(tidyverse)
library(rvest)
library(janitor)

# Step 2: Load data from Wikipedia ----
## Needed data: Wikipedia page for busiest passenger airports

airports_url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

# Step 3: Read the tables from the page ----

airport_tables <- airports_url |>
  read_html() |>
  html_elements("table.wikitable") |>
  html_table()

# Step 4: Keep the six yearly tables we need ----
## These are the annual tables for 2025, 2024, 2023, 2022, 2021, and 2020

years <- c(2025, 2024, 2023, 2022, 2021, 2020)
airport_tables_2020_2025 <- airport_tables[1:6]

# Step 5: Create a function to clean one yearly table ----

clean_airport_table <- function(one_table, one_year) {
  one_table |>
    as_tibble() |>
    clean_names() |>
    select(
```

```r
      code = matches("^code"),
      total_passengers = matches("total.*passengers")
    ) |>
    mutate(
      year = one_year,
      iata = substr(code, 1, 3),
      total_passengers = parse_number(as.character(total_passengers))
    ) |>
    select(year, iata, total_passengers)
}

# Step 6: Clean and stack all yearly tables ----

table_2025 <- clean_airport_table(airport_tables_2020_2025[[1]], 2025)
table_2024 <- clean_airport_table(airport_tables_2020_2025[[2]], 2024)
table_2023 <- clean_airport_table(airport_tables_2020_2025[[3]], 2023)
table_2022 <- clean_airport_table(airport_tables_2020_2025[[4]], 2022)
table_2021 <- clean_airport_table(airport_tables_2020_2025[[5]], 2021)
table_2020 <- clean_airport_table(airport_tables_2020_2025[[6]], 2020)

airportsCleaned <- bind_rows(
  table_2025,
  table_2024,
  table_2023,
  table_2022,
  table_2021,
  table_2020
)

# Step 7: Keep only the six airports for the assignment ----

airportsCleaned <- airportsCleaned |>
  filter(iata %in% c("ATL", "FRA", "LAX", "DXB", "DFW", "HND"))

# Step 8: Add readable airport names ----

airportsCleaned <- airportsCleaned |>
  mutate(
    airport = case_when(
      iata == "ATL" ~ "Hartsfield-Jackson Atlanta International Airport",
      iata == "FRA" ~ "Frankfurt Airport",
      iata == "LAX" ~ "Los Angeles International Airport",
      iata == "DXB" ~ "Dubai International Airport",
      iata == "DFW" ~ "Dallas Fort Worth International Airport",
      iata == "HND" ~ "Tokyo Haneda Airport"
    )
  )
```

```r
# Step 9: Arrange the data ----

airportsCleaned <- airportsCleaned |>
  select(year, airport, iata, total_passengers) |>
  arrange(year, airport)

### Creating the table necessary for the plot and the goals

# Create table for busiest airports ----

airportsTable <- airportsCleaned |>
  select(airport, year, total_passengers) |>
  pivot_wider(
    names_from = year,
    values_from = total_passengers
  ) |>
  arrange(airport)

airportsTable_display <- airportsTable |>
  mutate(
    airport = c(
      "Dallas Fort Worth (DFW)",
      "Dubai (DXB)",
      "Frankfurt (FRA)",
      "Atlanta (ATL)",
      "Los Angeles (LAX)",
      "Tokyo Haneda (HND)"
    )
  ) |>
  mutate(
    `2020` = format(`2020`, big.mark = ","),
    `2021` = format(`2021`, big.mark = ","),
    `2022` = format(`2022`, big.mark = ","),
    `2023` = format(`2023`, big.mark = ","),
    `2024` = format(`2024`, big.mark = ","),
    `2025` = format(`2025`, big.mark = ",")
  )

### Creating the plot for identifying trends

# Create plot for busiest airports ----

airportsPlot <- ggplot(data = airportsCleaned, aes(x = year, y = total_passengers, color = airp
  geom_line(size=1) +
  geom_point() +
  labs(
    title = "Passenger Traffic for Six Busiest Airports, 2020 to 2025",
```

```r
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  )

# Show plot
airportsPlot
```

6.2 Monte Carlo Simulation Final Code

```r
# Monte Carlo Numerical Integration Small Multiple Code ----
## Our goals are:
## - Create four Monte Carlo plots at different resolutions
## - Compare how the estimate changes as the number of points increases
## - Combine the four plots into one small multiple


### Wrangling the data and creating the plots

# Step 1: Load packages ----
## Needed packages: {tidyverse}, {ggplot2}, {patchwork}

# install.packages("patchwork")
library(tidyverse)
library(ggplot2)
library(patchwork)

# Step 2: Generate points for each resolution ----

generate_points <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x,
    y = y
  )
}

points10 <- generate_points(
  n = 10,
  x_min = 0,
  x_max = 20,
  y_min = 0,
  y_max = 0.16
```

```r
)

points100 <- generate_points(
  n = 100,
  x_min = 0,
  x_max = 20,
  y_min = 0,
  y_max = 0.16
)

points1000 <- generate_points(
  n = 1000,
  x_min = 0,
  x_max = 20,
  y_min = 0,
  y_max = 0.16
)

points10000 <- generate_points(
  n = 10000,
  x_min = 0,
  x_max = 20,
  y_min = 0,
  y_max = 0.16
)

# Step 3: Flag points relative to the curve ----

points10 <- points10 |>
  mutate(
    curve = dchisq(x, df = 5),
    under_curve = y <= curve
  )

points100 <- points100 |>
  mutate(
    curve = dchisq(x, df = 5),
    under_curve = y <= curve
  )

points1000 <- points1000 |>
  mutate(
    curve = dchisq(x, df = 5),
    under_curve = y <= curve
  )

points10000 <- points10000 |>
```

```r
  mutate(
    curve = dchisq(x, df = 5),
    under_curve = y <= curve
  )

# Step 4: Calculate Monte Carlo estimates ----

estimate10 <- (sum(points10$under_curve) / nrow(points10)) * ((20 - 0) * (0.16 - 0))
estimate100 <- (sum(points100$under_curve) / nrow(points100)) * ((20 - 0) * (0.16 - 0))
estimate1000 <- (sum(points1000$under_curve) / nrow(points1000)) * ((20 - 0) * (0.16 - 0))
estimate10000 <- (sum(points10000$under_curve) / nrow(points10000)) * ((20 - 0) * (0.16 - 0))

# Step 5: Create one plot for each resolution ----

plot10 <- ggplot(data = points10, aes(x = x, y = y)) +
  geom_point(aes(color = under_curve), alpha = 0.5) +
  stat_function(
    fun = dchisq,
    args = list(df = 5),
    xlim = c(0, 20),
    inherit.aes = FALSE
  ) +
  labs(
    title = "MonteCarlo Sim, n = 10",
    subtitle = paste("Estimate:", round(estimate10, 4)),
    x = "x",
    y = "y",
    color = "Under Curve"
  )

plot100 <- ggplot(data = points100, aes(x = x, y = y)) +
  geom_point(aes(color = under_curve), alpha = 0.5) +
  stat_function(
    fun = dchisq,
    args = list(df = 5),
    xlim = c(0, 20),
    inherit.aes = FALSE
  ) +
  labs(
    title = "MonteCarlo Sim, n = 100",
    subtitle = paste("Estimate:", round(estimate100, 4)),
    x = "x",
    y = "y",
    color = "Under Curve"
  )

plot1000 <- ggplot(data = points1000, aes(x = x, y = y)) +
```

```r
  geom_point(aes(color = under_curve), alpha = 0.5) +
  stat_function(
    fun = dchisq,
    args = list(df = 5),
    xlim = c(0, 20),
    inherit.aes = FALSE
  ) +
  labs(
    title = "MonteCarlo Sim, n = 1000",
    subtitle = paste("Estimate:", round(estimate1000, 4)),
    x = "x",
    y = "y",
    color = "Under Curve"
  )

plot10000 <- ggplot(data = points10000, aes(x = x, y = y)) +
  geom_point(aes(color = under_curve), alpha = 0.5) +
  stat_function(
    fun = dchisq,
    args = list(df = 5),
    xlim = c(0, 20),
    inherit.aes = FALSE
  ) +
  labs(
    title = "MonteCarlo Sim, n = 10000",
    subtitle = paste("Estimate:", round(estimate10000, 4)),
    x = "x",
    y = "y",
    color = "Under Curve"
  )

### Combining the four plots into a small multiple

# Step 6: Combine plots with patchwork ----

small_multiple_plot <- plot10 + plot100 + plot1000 + plot10000

# Show small multiple
small_multiple_plot
```